

Boltzmann Brain: A Versioned, Distributable, and Model-Agnostic Knowledge Architecture

Alex Fiorenza
alexfiorenza2012@gmail.com
Gaussia

github.com/gaussia-labs/papers

[] gaussia.ai/papers

huggingface.co/gaussia-labs

Abstract

Knowledge produced with large language models (LLMs) tends to become locked inside a single provider, agent, or vector store, where it is hard to audit, reindex, version, or move. We propose **Boltzmann Brain**, a portable knowledge architecture built on one principle: the brain conserves, validates, and retrieves knowledge, while an external and interchangeable LLM processes, contextualizes, and uses it. The brain never embeds a model of its own. Knowledge is stored as typed, content-addressed blocks organized into five memory modules (canonical, episodic, semantic, procedural, and provenance), inspired by complementary learning systems and hippocampal indexing rather than by literal brain-to-database mappings. Each module bundles its blocks, a Merkle DAG that pins the exact composition of a version, and the indices needed to query it. We separate three levels of hashes, one for block identity, one for the Merkle root of a snapshot, and one for the OCI digest of a distributed blob, and show how packaging each module as a separate OCI Artifact yields selective installation and incremental updates. Interaction happens through the *Boltzmann Protocol*: during ingestion and query the LLM proposes candidate blocks and consumes Evidence Bundles, while the protocol governs what is committed and returns only verified evidence, never prose. Because the brain is portable data addressed by a protocol rather than a deployable service, the same snapshot can be read and extended by any conforming client. This first version fixes the boundaries and responsibilities of the architecture and leaves concrete block schemas, wire formats, and algorithms for later validation.

Keywords: knowledge representation, retrieval-augmented generation, content addressing, Merkle DAG, OCI Artifacts, memory systems, LLM agents

1 Introduction

Large language models have made it cheap to *produce* knowledge (notes, summaries, extracted facts, procedures), but expensive to *own* it. In practice, the knowledge a user accumulates while working with an LLM tends to be captured in a form dictated by whichever provider, agent framework, or vector database happens to be in use. Embeddings are computed with a specific model, chunks are shaped by a specific pipeline, and the resulting store is rarely portable, verifiable, or re-interpretable. When the model changes, the provider changes, or the indexing strategy changes, the knowledge is often effectively lost, or must be rebuilt from scratch.

This paper argues that knowledge should be a *first-class, portable artifact* that outlives any particular model or vendor. We present **Boltzmann Brain**, an architecture whose sole responsibility is to conserve information, organize it into memories with distinct purposes, maintain

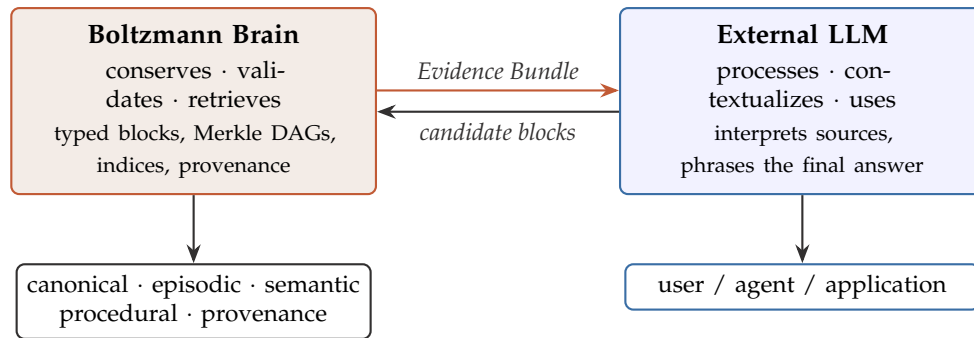


Figure 1: Separation of responsibilities. The brain owns knowledge, identity, and verification; the external LLM owns interpretation and phrasing. The two communicate through a stable contract: the LLM sends candidate blocks during ingestion and receives Evidence Bundles during query.

verifiable versions, and retrieve relevant evidence. Crucially, the brain contains no language model of its own. It integrates with whatever LLM the user chooses, which interprets sources during ingestion and contextualizes results during query. The design is captured by a single guiding idea:

The brain conserves, validates, and retrieves knowledge. An external LLM processes, contextualizes, and uses it.

The responsibility boundary. The brain does not write the final answer. It returns knowledge blocks and evidence; an agent, interface, or downstream application decides how to contextualize them (Figure 1). This boundary is what keeps the architecture independent of any provider: the Boltzmann Protocol governs structure, identity, validation, and distribution, while the semantic work (reading, extracting concepts, recognizing episodes, proposing procedures, and phrasing answers) is delegated to an interchangeable external model.

Contributions. This work makes the following contributions:

1. A set of design principles and problems (Section 2) for portable, verifiable, model-agnostic knowledge.
2. A mapping from biological memory principles and classic systems structures to concrete architectural decisions (Section 3).
3. A five-module memory model (canonical, episodic, semantic, procedural, and provenance; Section 5) together with an internal module anatomy of content-addressed blocks, Merkle DAGs, and indices (Section 6).
4. A distribution model based on OCI Artifacts that enables selective installation and incremental updates (Section 7), and a model-agnostic protocol with explicit ingestion and query contracts (Sections 8–10).

Scope of this version. This document fixes an initial conceptual architecture. It deliberately does not close the exact schema of each block, the consolidation algorithm, the size of packs, or the concrete index engine. The goal is to establish boundaries and responsibilities that later, more concrete work can validate.

2 Design Principles and Problems

2.1 Problems this architecture addresses

Boltzmann Brain is designed to address a cluster of related failures in how LLM-era knowledge is currently stored and consumed:

- Preventing knowledge from being trapped inside a particular provider, agent, or vector database.
- Maintaining a source of evidence that allows knowledge to be reindexed and reinterpreted as models and methods evolve.
- Distinguishing concrete experiences, general facts, and procedures, rather than collapsing everything into undifferentiated chunks.
- Updating knowledge without losing history, provenance, or reproducibility.
- Distributing a complete brain or only selected modules.
- Allowing interchangeable external LLMs to query and contribute knowledge through a stable contract.

2.2 Guiding principles

Seven principles follow from these problems and constrain every subsequent design decision:

1. Knowledge lives in blocks; indices are not the source of truth.
2. Original evidence must be preserved; interpretations may change.
3. Every version must be identifiable, verifiable, and reconstructible.
4. Modules must be distributable and installable selectively.
5. The protocol must not embed an LLM of its own nor depend on a specific provider.
6. Specialized structures compose; no single index answers every query.
7. Queries are declarative and index-agnostic: callers ask for knowledge, and the selection and combination of indices is left to the implementation.

These principles are intentionally conservative. They privilege durability and auditability over convenience, and they treat the LLM as a powerful but replaceable interpreter rather than as the system of record.

3 Inspirations

3.1 Biology of memory

The architecture takes *general principles* from neuroscience, not literal equivalences between brain regions and databases. The scientific literature describes multiple memory systems and an interaction among fast learning, stable knowledge, consolidation, associative recall, and adaptive forgetting [4, 6, 7, 10, 14–16, 20]. Figure 2 summarizes the translation from biological principles to architectural decisions.

Complementary learning systems (CLS) theory distinguishes a fast, hippocampus-associated mechanism from slower, integrative cortical learning [7]. For Boltzmann Brain this suggests

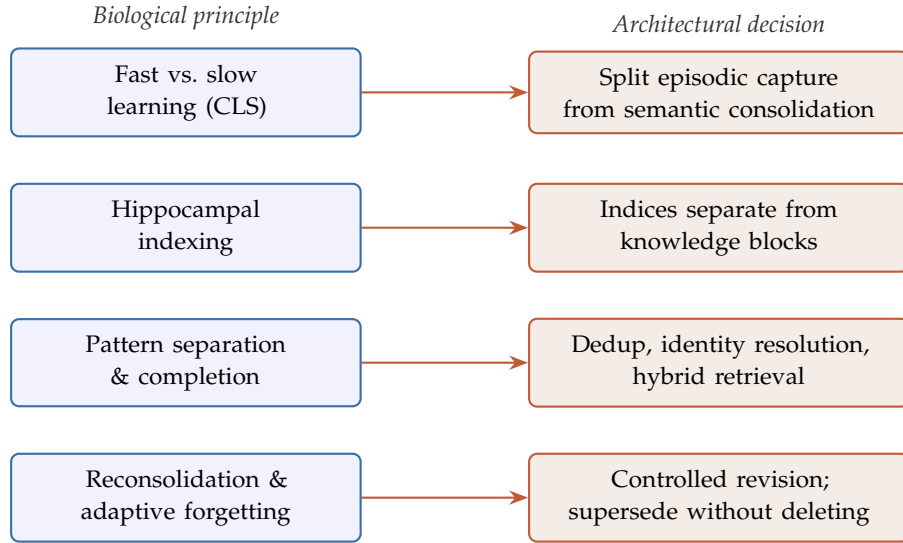


Figure 2: Translation of biological memory principles into architectural decisions. The mapping is by analogy, not by literal correspondence between brain regions and system components.

separating the *capture of episodes* from *semantic consolidation*. The hippocampal indexing theory further motivates a distinction between the index that reactivates a memory and the distributed content of that memory [4], a distinction we mirror in our separation of indices from knowledge blocks. Pattern separation inspires deduplication, identity resolution, and contradiction detection; pattern completion inspires recovering a full memory from a partial query through hybrid search [6, 14, 20]. Finally, reconsolidation suggests that retrieving knowledge can open a controlled revision, while adaptive forgetting suggests reducing accessibility without necessarily destroying the underlying evidence [10, 15, 16].

3.2 Systems structures

Alongside biology, the architecture reuses well-understood systems structures. Table 1 lists each one and its contribution.

Table 1: Systems structures and their contribution to Boltzmann Brain.

Concept	Contribution to Boltzmann Brain
Content addressing	The identity of a block derives from its content.
Merkle DAG	A root identifies the exact composition of a module.
B ⁺ tree	Exact lookup, hierarchies, and ordered range scans.
Inverted index	Lexical search over words and terms.
Vector index	Approximate retrieval by semantic similarity.
Graph	Explicit relations, provenance, associative expansion.
OCI Artifact	Interoperable packaging and distribution of modules.
Skill / MCP / CLI / API	Contract for an external LLM to query and ingest.

4 Related Work

The industry is already solving pieces of this problem, but no single artifact yet integrates a canonical source, typed memories, fine-grained versioning, portable indices, modular distribu-

tion, and a model-agnostic access protocol. Table 2 positions the most relevant efforts; we then discuss them in turn.

Table 2: Related efforts and their distance from Boltzmann Brain.

Project / standard	What it contributes	Distance from Boltzmann Brain
Open Knowledge Format (Google, 2026)	Markdown + YAML as a portable, human-readable format for agents.	Formalizes canonical exchange, but defines no access protocol, biological memory model, indices, or compiled distribution.
GBrain	Git repository as a durable registry; hybrid search, graph, and agent access.	Reinforces the independence of knowledge base from agent; its primary distribution follows the Git/repository model.
Microsoft GraphRAG	Extracts entities, relations, communities, and summaries; combines graph and text.	An indexing and retrieval pipeline, not a distributable format for a complete brain.
Cognee	Turns data into chunks, embeddings, nodes, and relations; combines vector and graph.	Provides memory operations and enrichment, but does not define the proposed OCI/Merkle bundle.
MCP	Client-server standard for tools, resources, and prompts.	A consumption interface; it defines neither storage nor versioning.
OCI / ORAS	Content-addressed artifacts, manifests, blobs, and registries.	Solves distribution; Boltzmann adds module semantics and an internal Merkle.

The Open Knowledge Format (OKF) formalizes the knowledge-wiki pattern through Markdown and YAML, emphasizing independence between producers and consumers [5]. GBrain separates durable knowledge from agent behavior and offers interchangeable engines and agent integration [17–19]. GraphRAG and Cognee illustrate the convergence toward hybrid representations that mix text, vectors, and graphs [2, 3, 8, 9]. MCP standardizes how AI applications connect to external resources and tools [1]. Finally, the OCI ecosystem provides content-addressed manifests, descriptors, and blobs whose structure is itself an external Merkle DAG [11–13].

Boltzmann Brain is complementary rather than competing with these efforts. It can consume OKF-style canonical sources, borrow hybrid retrieval ideas from GraphRAG and Cognee, expose an MCP interface, and ride on OCI for distribution, while contributing the missing piece: a versioned, typed, model-agnostic *bundle* whose modules can be installed and updated selectively.

5 Knowledge Modules

The brain’s OCI Artifact can contain five logical modules. Each module is installable as an independent physical unit and contains blocks, a Merkle DAG, and the indices needed for query. Table 3 names each module by the question it answers.

Table 3: The five memory modules, illustrated with a university brain.

Module	Question it answers	Examples
Canonical	What material was observed?	Notes, books, PDFs, lectures, code, datasets.
Episodic	What happened in a concrete context?	Class of May 14, an error in an exercise, a professor’s correction.
Semantic	What general knowledge was consolidated?	Definitions, formulas, facts, relations.
Procedural	How is a task performed?	Solving an integral, computing Fourier coefficients.
Provenance	Where did it come from and how was it transformed?	Page, source, pipeline, model, validator, date.

Canonical module. Preserves evidence. It may include original files and normalized representations, always maintaining a verifiable path back to the origin. It does not automatically declare a statement true: it declares that a source *contains* certain information.

Episodic module. Records events and experiences with time, context, participants, outcome, and evidence. It is naturally append-only: corrections generate new episodes or supersession relations rather than in-place edits.

Semantic module. Holds consolidated general knowledge: concepts, formulas, facts, relations, and constraints. A Fourier-series formula belongs primarily here, linked to its canonical sources.

Procedural module. Represents action sequences, conditions, decisions, alternative paths, and success criteria, for example how to obtain the Fourier coefficients of a given function.

Provenance module. Connects derived blocks to evidence, transformations, and tool versions. It is key to answering what must be recomputed when a source changes.

6 Anatomy of a Module

A module carries three responsibilities that coexist in the same physical unit (Figure 3): *blocks* contain knowledge; the *Merkle DAG* defines which blocks form a version; *indices* make it possible to find blocks without scanning all of them.

6.1 Knowledge blocks

A block is a logical unit of knowledge with a deterministic serialization. It may be an episode, a concept, a fact, a relation, a procedure, or a canonical fragment. Its identifier is computed from its content:

$$\text{block_id} = \text{SHA-256}(\text{canonical_serialization}(\text{block})). \quad (1)$$

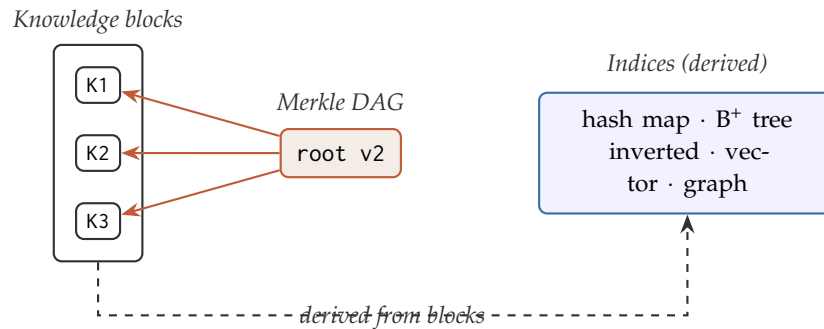


Figure 3: Blocks, a Merkle DAG, and indices coexist within the same module. The Merkle root pins the exact set of blocks that make up a version; indices are derived and can be rebuilt from the blocks.

If the content changes, a new block is born. The previous one may be kept for history, audit, or rollback.

6.2 Merkle DAG

On their own, blocks are just a bag of content-addressed units: each is addressable by its hash, but nothing names, verifies, or diffs a whole *version* of the module. The Merkle DAG solves exactly this. It stores no meaning by itself; it maintains references by hash and produces a single root that commits to the exact set and structure of blocks in a version. That root *is* the identity of the version, and it is what makes Principle 3 (identifiable, verifiable, and reconstructible versions) concrete.

How it works. The leaves of the DAG are `block_ids` (content hashes); each internal node hashes the hashes of its children; the root hashes down to everything below it. Changing any block changes its hash, which changes every ancestor up to the root. Three operations follow directly. *Integrity* is checked by recomputing hashes from the leaves up and comparing the root. *Membership* of a single block is proven with the sibling hashes along its path to the root (a Merkle proof of size $O(\log n)$), without downloading the rest of the module. *Differencing* two versions compares roots top-down and descends only where child hashes differ, so the cost is proportional to what changed, not to the size of the module.

Why a DAG rather than a tree. Because blocks are shared across versions. When a new version reuses unchanged blocks, their subtrees are shared by hash instead of duplicated, so a node may be referenced by more than one root; the structure is therefore a directed acyclic graph, as in Git’s object store. This structural sharing is what makes incremental updates cheap (Section 7): a new version adds only the changed blocks and a new root, and reuses everything else by reference.

Example

`semantic-root v1` contains K1, K2, and K3. If K2 is corrected, a new K2’ is created and `semantic-root v2` references K1, K2’, and K3. K2 is never modified in place, and K1 and K3 are shared between the two roots (Figure 4).

Two Merkle DAGs. It is worth separating the *internal* Merkle DAG described here, which versions the blocks inside a module, from the *external* Merkle DAG that OCI itself forms over

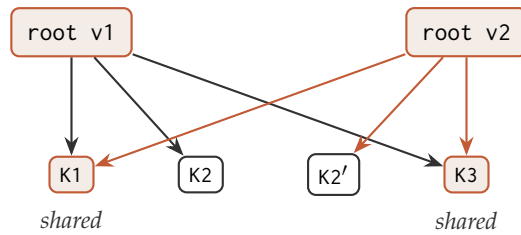


Figure 4: Structural sharing in the internal Merkle DAG. Correcting K2 into K2' yields a new root v2 that reuses K1 and K3 by hash; only the changed block and the new root are added. Solid dark edges belong to v1, accent edges to v2.

manifests, descriptors, and blobs to version whole modules (Section 7, Figure 5). The same idea is applied at two levels: blocks within a module, and modules within a brain.

Storage-independent identity. Because the root depends only on the blocks and their structure, it is a canonical identity of a knowledge state, independent of how that state is stored or transported. Two parties that assembled the same blocks obtain the same root and can verify they hold identical knowledge; the OCI digest, by contrast, identifies a particular physical file or manifest.

6.3 Indices

Indices are derived: they can be rebuilt from the blocks. They may travel inside the same module so that each consumer need not recompute them, but they never replace the content. Table 4 lists the indices and their roles.

Table 4: Indices, their typical queries, and their role.

Index	Typical query	Role
Hash map	Get exactly sha256:ABC.	Immediate physical resolution.
B ⁺ tree	Episodes between two dates; concept by ID.	Ordered keys and range scans.
Inverted	Blocks containing “Fourier”.	Lexical matching.
Vector	“Decompose a periodic function into sines”.	Semantic similarity.
Graph	Related concepts, evidence, dependencies.	Associative traversal.
Bitmap / facets	semantic AND subject=signals AND current.	Efficient filter intersection.

Whether an index travels inside the module or is rebuilt on installation depends on whether it can be regenerated without a model. Structural indices (hash map, B⁺ tree, inverted, bitmap, and, when relations live on the blocks, the graph) are deterministic functions of the blocks and can be rebuilt cheaply by any client. The vector index is the exception: rebuilding it requires an embedding model, which a model-agnostic client does not carry. The vector index therefore travels with the module and records the embedding model and version that produced it, so that consumers can reproduce or compare it. This keeps the blocks as the single source of truth while acknowledging that one derived structure cannot be regenerated in isolation.

This separation has a representational reading. A block carries meaning in two portable, *symbolic* forms: its text and its explicit relations to other blocks, the edges that in aggregate

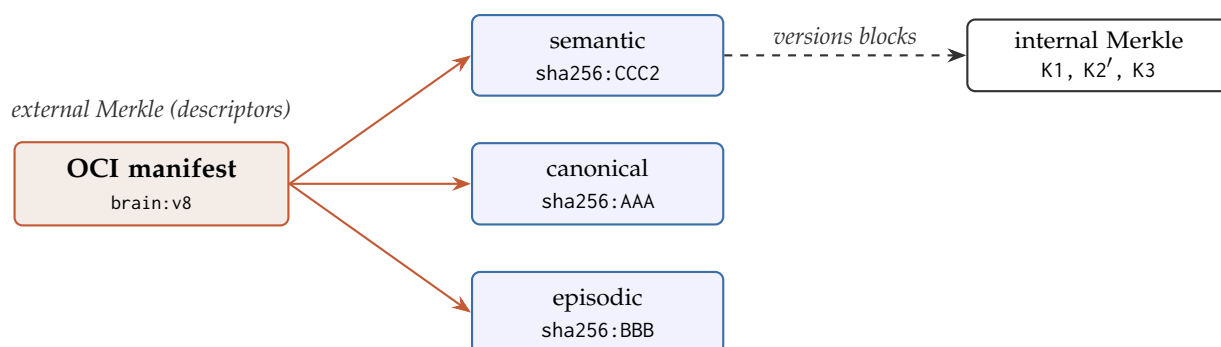


Figure 5: OCI distributes modules as separate blobs referenced by a small manifest; inside each module, an internal Merkle DAG versions the individual knowledge blocks.

form the knowledge graph. Learned, *sub-symbolic* representations (embeddings) are never stored in the block; they live in the vector index as derived, model-tagged views. Operating mathematically on concepts, such as vector arithmetic or steering, happens inside the external LLM in its own representation space, not in the brain. The brain thus owns durable symbolic knowledge and borrows the operational vector space from whichever model queries it. This also clarifies a naming coincidence: the *semantic* module denotes general, consolidated knowledge (as opposed to episodic memory), not an embedding-based representation.

6.4 Three levels of hashes

A recurring source of confusion is that the same cryptographic algorithm is used for three different kinds of identity. Table 5 separates them.

Table 5: Three levels of hashes.

Level	What it identifies	Example
Block hash	A logical unit of knowledge.	sha256:FOURIER_CONCEPT
Merkle root	The logical composition of a module.	sha256:SEMANTIC_ROOT_V7
OCI digest	The published physical blob or manifest.	sha256:OCI_MODULE_BLOB

These values may use the same algorithm, but they do not represent the same thing. The block hash is *knowledge identity*; the Merkle root is the identity of a *logical snapshot*; the OCI digest is the identity of a transportable *file or manifest*.

7 Distribution with OCI Artifacts

OCI is a standard for content-addressed manifests, descriptors, and blobs. A compatible registry stores the blobs and allows referencing them by tag or digest; the specification organizes its components as an external Merkle DAG of descriptors [11–13]. Figure 5 shows how OCI distributes modules while the internal Merkle versions the blocks within each module.

7.1 Complete brain

A brain can be published as an OCI Artifact that references one blob per module:

```

monotributo-brain:v3
  canonical -> sha256:AAA
  episodic  -> sha256:BBB
  
```

```

semantic    -> sha256:CCC
procedural  -> sha256:DDD
provenance  -> sha256:EEE

```

The OCI manifest is small. Downloading the manifest does not imply downloading all modules.

7.2 Selective installation

To install only the episodic module, the client resolves the descriptor marked as episodic and downloads sha256:BBB. It does not need to download canonical, semantic, or procedural. It then opens the episodic Merkle DAG and its local indices.

Necessary condition

Selection is only efficient if each module is a separate OCI blob or artifact. If everything is mixed into a single file, the whole file must be downloaded.

7.3 Incremental update

If only the semantic module changes, the new manifest can reuse the digests of the remaining modules. The consumer downloads only the new semantic blob; inside it, the Merkle root reveals which logical blocks changed.

Table 6: An incremental update touches only the changed module's digest.

Version	Canonical	Episodic	Semantic	Procedural
v7	AAA	BBB	CCC	DDD
v8	AAA	BBB	CCC2	DDD

OCI contributes storage, registry authentication, transport, digest-based deduplication, and tags. Boltzmann Brain contributes the meaning of the modules, the blocks inside them, and their query contract.

8 The Boltzmann Protocol

An installed OCI Artifact is passive data. The *Boltzmann Protocol* is the contract through which any conforming client interacts with a brain, along two paths, query and ingestion. Boltzmann Brain is therefore delivered as a protocol rather than as a deployable service or container image: the brain is portable data, and any client that speaks the protocol can read and extend it. The protocol includes no LLM of its own; it governs structure, identity, validation, indices, snapshots, and distribution. Concretely, it defines operations to:

- Discover installed modules, versions, and Merkle roots.
- Open B^+ , lexical, vector, and graph indices.
- Resolve a `block_id` to its physical location and verify its hash.
- Register canonical sources, provenance, and `derived_from` relations.
- Define processing tasks and output schemas for an external LLM.
- Validate candidate blocks before they are incorporated into the brain.

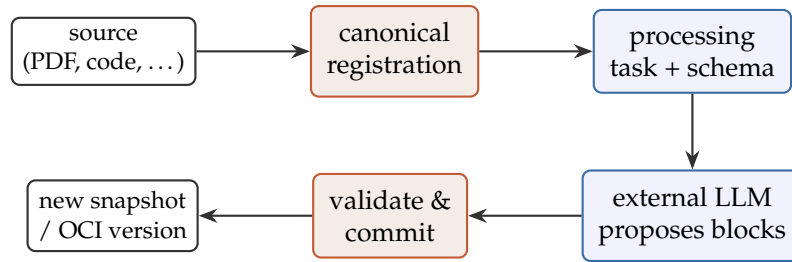


Figure 6: The brain conserves and validates; the external LLM interprets and proposes blocks. Only validated candidates are committed and versioned.

- Update Merkle DAGs, indices, and snapshots.
- Publish new versions as OCI Artifacts.
- Be exposed through a skill, MCP, CLI, or API.

Because these are protocol operations rather than a bundled engine, they can be implemented independently and in any language, and the brain never depends on a particular process being kept running.

8.1 The boundary between the protocol and the external LLM

The protocol defines which memory types exist and which schema each block must satisfy. The external LLM performs the semantic operations: reading, interpreting, extracting concepts, recognizing episodes, proposing procedures, and contextualizing results. *The LLM proposes; the protocol governs what is stored.*

This boundary keeps the brain provider-independent. The same snapshot can be used with a commercial service, a development agent, or a local model, as long as the client implements the Boltzmann Protocol. Trust does not rest on a privileged service: because every block, snapshot, and module is addressed by its hash, any client can verify integrity independently of who produced it.

Design rule

The LLM never writes directly to the Merkle DAGs or to the indices.

9 Ingestion via an External LLM

Ingestion separates the preservation of a source from its interpretation (Figure 6). First the original is registered deterministically; then the protocol delegates semantic processing to the user's LLM.

9.1 Canonical registration

1. The user provides a PDF, text, repository, conversation, image, or other source.
2. The LLM invokes the `brain.ingest` operation through the available skill or adapter.
3. The format is validated, its SHA-256 computed, and exact duplicates detected.
4. The original is stored as an immutable canonical block.

5. The protocol records who incorporated it, when, from where, and under what license or policy.
6. The canonical Merkle root changes and a new snapshot becomes available, even though no semantic interpretation exists yet.

Registering a source does not declare it true. The canonical module asserts that the evidence was incorporated and preserved.

9.2 Processing task

After canonical registration, the protocol returns a structured task to the LLM. The task identifies the source, the instructions, the allowed memory types, the required references, and the exact output schema:

```
{
  "operation": "extract_knowledge",
  "source": "sha256:PDF123",
  "allowed_memory_types": ["episodic", "semantic", "procedural"],
  "requirements": ["cite source ranges", "do not invent"],
  "output_schema": "boltzmann.candidates/v1"
}
```

The LLM reads the source and returns typed candidate blocks. A document may produce semantic and procedural memory without producing episodic memory; the classification depends on what the source actually contains.

9.3 Validation and commit

Results from the LLM do not enter automatically. The protocol checks schema, references, pages, types, duplicates, relations, and basic contradictions. Candidates may end up `VALIDATED`, `PENDING_REVIEW`, `REJECTED`, or `CONTRADICTED`. For each accepted block, the protocol prescribes the following steps:

1. Serializes the block canonically.
2. Computes its `block_id` by hashing.
3. Stores the block as an immutable object.
4. Connects it to the source via provenance.
5. Incorporates it into the episodic, semantic, or procedural Merkle DAG.
6. Updates B^+ trees, lexical, vector, and graph indices as appropriate.
7. Creates a new snapshot and, when published, a new OCI version.

10 Query via an External LLM

For now, Boltzmann Brain is conceived as a medium for LLMs. The user converses with their usual LLM; a skill or adapter lets that LLM query the brain through the protocol. The brain returns raw or semi-raw knowledge, and the LLM turns it into an answer appropriate for the task (Figure 7).

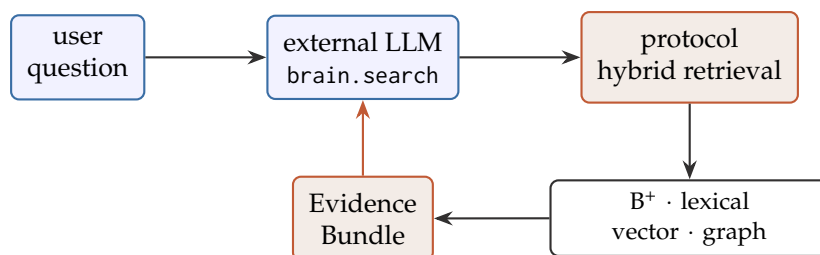


Figure 7: The external LLM queries the brain and contextualizes the returned Evidence Bundle. The protocol combines indices, verifies hashes, and returns data rather than prose.

10.1 Query flow

1. The user asks the external LLM a question.
2. The LLM interprets the intent and calls `brain.search` with the query and optional filters.
3. The protocol combines B^+ trees, lexical search, vectors, and graph.
4. Knowledge blocks and their canonical sources are retrieved.
5. The protocol verifies hashes and membership in the installed snapshot.
6. An Evidence Bundle is returned.
7. The LLM explains, summarizes, compares, or uses the evidence.

10.2 Query planning and guarantees

Step three above hides a query planner. By Principle 7 a query is declarative: the caller expresses intent (a query, optional filters over memory type, subject, or recency, and optional hints such as a retrieval mode or a result limit) and never names a physical index. Choosing which indices to use, and how to combine them, is the responsibility of a conforming implementation, in the same way that the SQL standard defines a query language and its semantics while each database ships its own optimizer. Selection follows the shape of the query: exact identifiers resolve through the hash map, ordered or range predicates through the B^+ tree, term matches through the inverted index, natural-language intent through the vector index, neighborhood and traversal through the graph, and categorical filters through bitmaps. In practice retrieval is hybrid: filters narrow a candidate set, lexical and vector search generate candidates in parallel, their rankings are fused, the graph expands from the strongest hits, and the hash map resolves and verifies each result.

The protocol fixes the contract and the invariants, not the algorithm. A conforming implementation must return knowledge blocks with their provenance and a retrieval score, never prose; every returned block must be verified against the installed snapshot by hash and membership; and no single index may be treated as authoritative. The fusion method, ranking weights, graph expansion depth, and any adaptive intent classification are deliberately left to the implementation. A consequence worth stating explicitly is that the protocol guarantees *verifiability, not identical ranking*: two conforming implementations may return the same verifiable set of blocks in a different order, because vector search is approximate and ranking is tunable.

10.3 Evidence Bundle

The brain's result is not a written answer. It is a data contract that may include matching blocks, content, memory type, relations, sources, retrieval score, block IDs, and verification status:

```
{
  "matches": [{
    "block_id": "sha256:CONCEPT1",
    "memory_type": "semantic",
    "content": "<serialized knowledge block content>",
    "source": {"block_id": "sha256:PDF123", "page": 147},
    "verified": true
  }]
}
```

The consumer receives block identities, content, memory type, sources, and verification status, and remains free to explain, summarize, compare, or reuse the evidence. Those transformations happen outside the brain.

11 Knowledge Lifecycle

The full lifecycle ties the previous sections together:

1. **Source:** the user provides information via their LLM and a skill.
2. **Canonical registration:** the protocol preserves the original, computes its hash, and records provenance.
3. **Delegation:** the protocol returns a task and a processing schema.
4. **Interpretation:** the LLM proposes episodic, semantic, and procedural blocks and relations.
5. **Validation:** the protocol checks structure, references, duplicates, and policies.
6. **Commit:** accepted candidates receive hashes and are stored as immutable blocks.
7. **Indexing:** B^+ trees, lexical search, vectors, and graph are updated.
8. **Versioning:** the Merkle roots of the affected modules change.
9. **Publication:** OCI distributes only the modified physical modules.
10. **Query:** the LLM obtains Evidence Bundles and transforms them for the user.
11. **Reconsolidation:** new evidence or corrections generate later candidates and snapshots.
12. **Adaptive forgetting:** obsolete content can become superseded or lose priority without destroying the audit trail.

The canonical, episodic, semantic, procedural, and provenance memories remain at the center of the architecture. The change consists solely in decoupling cognitive processing: the protocol organizes and governs; the external LLM interprets and contextualizes.

12 Conclusion and Future Work

We presented Boltzmann Brain, a conceptual architecture for portable, verifiable, and model-agnostic knowledge. Its core commitment is a clean separation of concerns: the brain conserves, validates, and retrieves, while an external, interchangeable LLM interprets and contextualizes. From this commitment follow the five-module memory model, the content-addressed blocks

versioned by Merkle DAGs, the derived indices, the three levels of hashes, and a distribution model in which OCI Artifacts enable selective installation and incremental updates.

This version deliberately fixes boundaries and responsibilities rather than implementations. Concrete block schemas, wire formats, and algorithms remain to be validated: the exact block schema per memory type, the deterministic serialization used for `block_id`, the consolidation and contradiction-detection algorithms, the pack sizing and index engine choices, and the precise shape of the ingestion and query operations exposed through skills, MCP, CLI, or API. We also see promising work in formalizing supersession and adaptive forgetting so that accessibility can decay without compromising auditability. Validating these pieces against real, multi-source brains is the natural next step.

References

- [1] Anthropic. Model context protocol: Architecture overview. Model Context Protocol Specification, 2024. <https://modelcontextprotocol.io>.
- [2] Cognee. The cognify operation. Cognee Documentation, 2025. <https://docs.cognee.ai>.
- [3] Cognee. Core concepts overview. Cognee Documentation, 2025. <https://docs.cognee.ai>.
- [4] Travis D. Goode, Kazumasa Z. Tanaka, Amar Sahay, and Thomas J. McHugh. The hippocampal engram as a memory index. *Neurobiology of Learning and Memory*, 2018.
- [5] Google Cloud. Introducing the open knowledge format. Google Cloud Blog, 2026. <https://cloud.google.com/blog>.
- [6] James J. Knierim and Joshua P. Neunuebel. Tracking the flow of hippocampal computation: Pattern separation, pattern completion, and attractor dynamics. *Neurobiology of Learning and Memory*, 129:38–49, 2016.
- [7] Dharshan Kumaran, Demis Hassabis, and James L. McClelland. What learning systems do intelligent agents need? complementary learning systems theory updated. *Trends in Cognitive Sciences*, 20(7):512–534, 2016.
- [8] Microsoft. Graphrag: Indexing architecture and methods. Microsoft GraphRAG Documentation, 2024. <https://microsoft.github.io/graphrag>.
- [9] Microsoft. Graphrag: Query engine overview. Microsoft GraphRAG Documentation, 2024. <https://microsoft.github.io/graphrag>.
- [10] Karim Nader and Oliver Hardt. A single standard for memory: The case for reconsolidation. *Nature Reviews Neuroscience*, 10(3):224–234, 2009.
- [11] Open Container Initiative. Oci descriptor specification. OCI Specifications, 2024. <https://github.com/opencontainers/image-spec>.
- [12] Open Container Initiative. Oci image manifest specification. OCI Specifications, 2024. <https://github.com/opencontainers/image-spec>.
- [13] ORAS. Understanding oci artifacts. ORAS Documentation, 2024. <https://oras.land>.
- [14] Nelson Rebola, Mario Carta, and Christophe Mulle. Operation and plasticity of hippocampal ca3 circuits: Implications for memory encoding. *Nature Reviews Neuroscience*, 18(4): 208–220, 2017.

- [15] Roger L. Redondo and Richard G. M. Morris. Making memories last: The synaptic tagging and capture hypothesis. *Nature Reviews Neuroscience*, 12(1):17–30, 2011.
- [16] Tomas J. Ryan and Paul W. Frankland. Forgetting as a form of adaptive engram cell plasticity. *Nature Reviews Neuroscience*, 23(3):173–186, 2022.
- [17] Garry Tan. Brain vs memory vs session. GBrain, GitHub repository, 2026. <https://github.com/garrytan/gbrain>.
- [18] Garry Tan. Pluggable engine architecture. GBrain, GitHub repository, 2026. <https://github.com/garrytan/gbrain>.
- [19] Garry Tan. Two-repo architecture: Agent behavior vs world knowledge. GBrain, GitHub repository, 2026. <https://github.com/garrytan/gbrain>.
- [20] Michael A. Yassa and Craig E. L. Stark. Pattern separation in the hippocampus. *Trends in Neurosciences*, 34(10):515–525, 2011.